



Mi Max 2 Pie-ified?

An XDA senior member has release Android Pie custom ROMs for the Xiaomi Mi Max 2.
<https://dgit.in/dec18-44-1>



Pretend to work

A Google Chrome extension made by a Japanese programmer disguises Twitter as Slack so you can pretend to work. <https://dgit.in/dec18-44-2>

dresses only — e.g. 127.0.0.1. This is sufficient to run a single development node on a server. To form a cluster with nodes on other servers, the node will need to bind to a non-loopback address as shown below.

```
network.host: 192.168.1.10
```

Discovery settings

Elasticsearch uses a custom discovery algorithm called "Zen Discovery" for clustering and master assignment. There are a couple of important discovery settings that should be configured before going to production.

When the moment comes to form a cluster with nodes on other servers, you have to provide a seed list of other nodes in the cluster that are likely to be live and contactable. This can be specified as shown here: <https://dgit.in/ES3>.

It is also crucial to configure the `discovery.zen.minimum_master_nodes` setting so that each master-eligible node knows the minimum number of nodes that must be visible in order to form a cluster.

```
discovery.zen.minimum_master_nodes: 2
```

All the settings that can be specified in the config file can also be specified on the command line, using the -E syntax as follows:

```
./bin/elasticsearch -d
-Ecluster.name=my_cluster
-Enode.name=node_1
```

USING ELASTICSEARCH

Now, that we have elasticsearch running, and the majority of its configuration explained, lets try using it for indexing and searching documents.

Index a document

In order to index we need to create a json object and make a POST request to a URL made up of the index name, type name and ID. That is:

```
http://localhost:9200/<index>/<type>/<id>.
```

Index and type are mandatory while the id part is optional. If we don't specify an ID Elasticsearch will generate one for us.

The index name is root name for a document index, all documents of the same index name are indexed together. If there isn't an index with that name on the server already one will be created using default configuration.

The type name too is arbitrary. It serves several purposes, including:

- Each type has its own ID space.
- Different types can have different mappings ("schema") that defines how properties/fields should be indexed).

Let us now index a document into elasticsearch, as shown here: <https://dgit.in/ES4>. This command would index a document with the above data at index 'songs' with type 'rock' and id '1'.

To verify, simply search for all documents in elasticsearch as follows:

```
curl "http://localhost:9200/songs/_search"
```

The output should contain your indexed document. Congratulations! You have successfully indexed your first document in elasticsearch.

Get a document

In order to fetch a single document using its ID we need to make a GET request to the same URL as when we indexed it, only this time the ID part of the URL is mandatory. i.e. <http://localhost:9200/<index>/<type>/<id>>.

```
curl "http://localhost:9200/songs/rock/1"
```

The result object returned contains an index, type and version information. Finally, it also has a property named "_source" which contains the actual document.

Deleting documents

In order to remove a single document from the index by ID we again use the same URL as getting it, only this time we change the HTTP method to DELETE.

```
curl -XDELETE "http://localhost:9200/movies/movie/1"
```

The response object here contains a property named "_found" that tells if the document was indeed found and that the operation was successful.

*POINTERS



*Getting Started with Elasticsearch

Learn basics of Elasticsearch from scratch with this 101 course on Udemy and become an Elasticsearch Padawan.
<https://dgit.in/ESudemy>



*Become an Elastic Certified Engineer

The Elastic Stack is versatile enough to tackle any use case. These certification programs show you how to become an Elastic expert.
<https://dgit.in/ElasticCertified>



*An Elasticsearch Crash Course

If you've ever wondered how full text search engines can rip through and sort a mountain of data, this is the crash course for you to understand that.
<https://dgit.in/ElasticCCourse>